

Donuts

```
package main

import (
    "fmt"
    "math/rand"
    "os"
    "os/exec"
    "time"
)

type donut string
var donut_flavours []string = []string{"Chocolate Topping",
"Jam Filled", "Custard Filled", "Millions Topping", "Plain
Ring"}
var donut_ascii string =
`
```



```
func donut_menu() {  
    fmt.Printf("\n      Ready to bake a donut! Press  
enter!\n")  
    fmt.Scanf("%s")  
}  
  
func bake_a_donut() donut {  
    for _, char := range donut_ascii {  
        fmt.Printf("%c", char)  
        time.Sleep(5 * time.Millisecond)  
    }  
    return "\n  A BEAUTIFUL PLAIN DONUT, FRESHLY BAKED"  
}  
  
func (d donut )add_a_flavour() {  
    clear_screen()  
    fmt.Printf("\n*** SELECTING A FLAVOUR ***")  
    time.Sleep(3000 * time.Millisecond)  
    count := 50  
  
    for count > 0 {  
        for i := range len(donut_flavours) {  
            fmt.Println(donut_flavours[i])  
            time.Sleep(100 * time.Millisecond)  
            clear_screen()  
            count--  
        }  
    }  
  
    rand.Seed(time.Now().UnixNano())  
    flavour := rand.Intn(len(donut_flavours))  
  
    fmt.Println(donut_ascii)
```

```
        fmt.Printf("    TODAY'S FLAVOUR IS = %s\n\n",
donut_flavours[flavour])
    }

func clear_screen() {
    cmd := exec.Command("clear")
    cmd.Stdout = os.Stdout
    cmd.Run()
}
```

```
package main

import (
    "fmt"
    "time"
)

func main() {

    donut_menu()
    todays_donut := bake_a_donut()
    fmt.Println(todays_donut)
    time.Sleep(3000 * time.Millisecond)
    todays_donut.add_a_flavour()
}
```